

Angular Moderno

Bienvenidos a la era de las SPA (Single Page Applications)

Un curso de 40 horas enfocado al desarrollo profesional real.

Objetivo: transformar la aplicación de coches de JavaScript puro (cochesModulosV2) en una plataforma web moderna.

Trabajaremos con herramientas y flujos habituales en empresas tecnológicas.

De páginas sueltas a una aplicación única

Antes, con JavaScript puro, teníamos coches.html, marcas.html y modelos.html. Al pulsar un enlace, el navegador borraba la pantalla, pedía otro archivo al servidor y lo pintaba de nuevo.

Ahora construiremos una SPA: una Single Page Application. Solo hay una página real en el proyecto: index.html.

Angular se encarga de quitar y poner piezas en pantalla sin recargar toda la página. Para el usuario, la aplicación se siente más rápida, fluida y continua.

Idea clave: cambia la arquitectura, no la aplicación. Seguiremos trabajando con coches, marcas y modelos.

Node.js y NPM: el taller local

Node.js

JavaScript nació para vivir dentro del navegador. Node.js permite ejecutarlo en nuestro ordenador, como cualquier otra herramienta de desarrollo.

NPM

NPM significa Node Package Manager. Se instala con Node.js y nos permite descargar paquetes creados por la comunidad.

Analogía

Node.js es el motor que permite arrancar las herramientas. NPM es el repartidor que trae las piezas: Angular, librerías, validadores y utilidades.

En la terminal diremos cosas como: `npm install -g @angular/cli`

Angular CLI y el esqueleto prefabricado

Angular CLI

Es la herramienta de comandos de Angular. Nos permite crear, ejecutar y mantener proyectos con órdenes simples como `ng new` o `ng serve`.

Scaffold o andamiaje

Angular necesita una estructura profesional: carpetas, configuración de TypeScript, estilos, scripts de arranque y pruebas. Crear todo eso a mano sería lento y propenso a errores.

`ng new`

Es como montar una casa prefabricada: coloca los cimientos del proyecto para que nosotros empecemos a construir las habitaciones, que en Angular llamaremos componentes.

Angular, TypeScript y JavaScript

Angular se programa con TypeScript.

TypeScript no sustituye a JavaScript: es JavaScript con una capa extra de tipos, reglas y ayuda del editor. Al final, el proyecto se compila y el navegador ejecuta JavaScript.

Para un principiante, la idea importante es esta:

- JavaScript es el idioma que entiende el navegador.
- TypeScript nos ayuda a escribir ese idioma con más seguridad.
- Angular usa TypeScript para organizar componentes, servicios, rutas y modelos de datos.

Ejemplo de modelo:

```
export interface Marca { id: number; nombre: string; }
```

Cliente, API y datos

En el proyecto JS puro:

Navegador -> fetch -> Supabase REST API -> tablas

En Angular será parecido:

Componente -> Servicio Angular -> HttpClient -> Supabase REST API -> tablas

Los datos siguen siendo marcas, modelos y coches.
Lo que cambia es dónde vive cada responsabilidad.

El componente se ocupa de la pantalla.

El servicio se ocupa de hablar con la API.

HttpClient es la herramienta de Angular para hacer peticiones HTTP.

Supabase sigue siendo el backend que guarda los datos.

Instalación

PASO 1

Instalación de Node.js

<https://nodejs.org/es>

Descargar la version LTS.

Comprobar en terminal:

```
node -v
```

```
npm -v
```

PASO 2

Instalar Angular CLI:

```
npm install -g @angular/cli
```

Comprobar:

```
ng version
```

Instalación

PASO 3

Creamos una carpeta de trabajo para el curso.

Ejemplo:

```
cd Desktop  
mkdir curso-angular  
cd curso-angular
```

Creamos el proyecto:

```
ng new coches-angular
```

Opciones recomendadas para empezar:

– CSS

- Routing: podemos decir Yes si ya queremos preparar rutas.

Este proyecto será nuestro hola mundo y la base del CRUD.

Instalación

PASO 4

Entramos en el proyecto y arrancamos el servidor:

```
cd coches-angular  
ng serve --open
```

Si no abre automáticamente:
<http://localhost:4200>

El servidor observa los archivos.

Cuando guardamos cambios, Angular recompila y actualiza la aplicación.

Primeros pasos

Carpetas importantes:

`node_modules`

Dependencias instaladas por npm. No se edita manualmente y no se sube a GitHub.

`src`

Código fuente que trabajaremos.

`src/app`

Zona principal de componentes, rutas, servicios y modelos.

`package.json`

Lista de scripts y dependencias del proyecto.

Primeros pasos

En Angular no escribimos una página HTML completa dentro de cada componente.

En el componente ponemos solo el contenido de esa parte de pantalla.

Ejemplo inicial en app.html:

```
<h1>CRUD de coches en Angular</h1>  
<p>Entorno Angular preparado.</p>
```

Primeros pasos

index.html contiene la estructura general del documento.

Dentro aparece:

```
<app-root></app-root>
```

Angular coloca ahí el componente principal.

Idea importante:

index.html es la puerta de entrada.

app.html es lo que vemos dentro de la aplicación.

Primeros pasos

Archivos principales:

app.html

Plantilla HTML del componente.

app.css

Estilos del componente.

app.ts

Clase TypeScript del componente.

app.spec.ts

Pruebas. De momento no lo usaremos.

Primeros pasos

app.ts

Aquí vive la clase del componente.

Ejemplo conceptual:

```
export class AppComponent {  
  titulo = signal('CRUD de coches en Angular');  
  descripcion = signal('Migración desde JS puro');  
}
```

Después podemos mostrarlo en HTML:

```
<h1>{{ titulo }}</h1>
```

Muy importante

Nombres de archivos:

app.ts
app.html
app.css

Nombre de la clase:

App

Convención:

- archivos en minúsculas;
- clase en PascalCase;
- componentes con nombres claros.

No buscamos memorizar todo hoy.
Buscamos reconocer la estructura.

Archivos de arranque

main.ts

Arranca la aplicación Angular.

styles.css

Estilos globales de toda la aplicación.

app.config.ts

Configuración general: router, HttpClient y otros proveedores.

assets o public

Recursos estáticos: imágenes, iconos, fuentes, etc.

Angular 1

Ejercicio rápido

1. Crear el proyecto coches-angular.
2. Arrancarlo con `ng serve --open`.
3. Modificar `app.html`.
4. Escribir un título y una frase sobre el CRUD.
5. Comprobar que el navegador se actualiza.

Resultado:

El entorno Angular funciona.

Instalación en Mac

Si aparece un problema de permisos en Mac, no copiar comandos destructivos sin entender el error.

Primero comprobar:

node -v

npm -v

npm config get prefix

Soluciones habituales:

- **usar instalador oficial de Node LTS;**
- usar nvm para gestionar Node;
- cerrar y abrir terminal;
- revisar permisos solo sobre la carpeta afectada.

sudo chown solo debe usarse si sabemos exactamente qué carpeta falla.

Crear proyecto real

Aunque el ejemplo se parezca a holaMundo, el proyecto del curso será:

coches-angular

Comando:

ng new coches-angular

Por qué no crear otro ejemplo separado:

- ahorramos tiempo;
- todos trabajan sobre la base real;
- el primer cambio ya pertenece al proyecto que evolucionará.

Servidor de desarrollo

Comandos útiles:

```
cd coches-angular  
ng serve --open
```

Si el puerto 4200 está ocupado:

```
ng serve --port 4201 --open
```

Para parar el servidor:

Ctrl + C

Cada vez que abrimos el proyecto en otro día:

```
cd coches-angular  
npm install (solo si faltan dependencias)  
ng serve --open
```

Cursor y Git

Abrimos la carpeta coches-angular con Cursor.

Primeras reglas:

- no editar node_modules;
- leer antes de cambiar;
- pedir a la IA explicaciones pequeñas;
- no aceptar código que no entendemos.

Primer versionado:

```
git init
git add .
git commit -m "Crear proyecto Angular inicial"
git tag v0.1
```

Angular 2

Ejercicio de versionado

1. Abre el proyecto en Cursor.
2. Cambia el texto inicial de app.html.
3. Comprueba el navegador.
4. Haz commit.
5. Crea el tag v0.1.

Objetivo:

igual que en JS puro versionamos por hitos, en Angular también cerraremos versiones.

Qué haremos en el curso

Partimos de la especificación ya creada.

Ruta del curso:

1. Leer especificación y arquitectura.
2. Crear estructura Angular.
3. Traducir tablas a interfaces TypeScript.
4. Crear componentes.
5. Crear servicios para Supabase.
6. Construir formularios.
7. Añadir rutas.
8. Revisar, refactorizar y versionar.

JS puro -> Angular

Equivalencias que veremos:

coches.html, marcas.html, modelos.html -> rutas y componentes

innerHTML/createElement -> templates

addEventListener -> eventos Angular

fetch -> HttpClient

ListaCoches.js -> servicio + componentes

spec.md -> guía de desarrollo

Git tags -> versiones del proyecto

La especificación sigue mandando.

Trabajo para casa

Leer el proyecto JS puro y responder:

1. ¿Qué módulos tiene la aplicación?
2. ¿Qué datos tiene una marca?
3. ¿Qué datos tiene un modelo?
4. ¿Qué datos tiene un coche?
5. ¿Qué reglas hay entre marca, modelo y coche?
6. ¿Qué parte del código JS parece más difícil de mantener?

La próxima clase empezaremos a traducir esto a Angular.

Cierre

Hoy deberíamos tener:

- Node instalado.
- Angular CLI instalado.
- Proyecto coches-angular creado.
- Servidor funcionando en localhost.
 - Primer cambio visible.
- Proyecto abierto en Cursor.
 - Primer commit.
 - Primer tag v0.1.

Siguiente paso:
convertir la especificación en estructura Angular.